

检索增强生成（RAG）技术详解

1. RAG 技术概述

什么是 RAG

检索增强生成（Retrieval-Augmented Generation, 简称 RAG）是一种将外部知识检索与大语言模型（LLM）生成能力相结合的技术范式。其核心在于，在模型生成回答之前，先从外部知识库中检索与用户查询相关的信息，并将这些信息作为上下文输入给 LLM，从而引导模型基于事实生成更准确、更可靠的回答。

为什么需要 RAG

尽管大语言模型在通用任务上表现卓越，但仍存在显著局限：- **知识时效性滞后**：模型训练数据截止于特定时间点，无法获取最新信息。- **领域知识匮乏**：通用模型缺乏企业私有数据或特定垂直领域的专业知识。- **幻觉问题**：模型可能生成看似合理但事实错误的内容。- **数据隐私与安全**：企业敏感数据不宜直接用于模型微调或训练。

RAG 通过“外挂大脑”的方式，在不重新训练模型的前提下，有效解决了上述问题，实现了知识的实时更新与私有化部署。

RAG 的核心思想

RAG 的核心思想可以概括为“先检索，后生成”。它将问题求解过程解耦为两个阶段：1. **检索阶段**：将用户自然语言查询转化为向量或关键词，在索引库中召回相关文档片段。2. **生成阶段**：将检索到的文档片段与原始查询拼接为 Prompt，输入 LLM，模型基于提供的上下文生成最终答案。

这种架构不仅降低了模型对参数化记忆的依赖，还使得答案具备可追溯性，极大提升了系统的可信度。

2. RAG 的系统架构

一个标准的 RAG 系统通常包含离线索引构建与在线查询生成两个流程，核心组件如下：

索引构建模块

- 文档预处理**：对非结构化数据（PDF、Word、HTML 等）进行解析、清洗与分块（Chunking）。分块策略直接影响检索粒度，常见策略包括固定长度切分、基于语义的切分及递归字符切分。
- 向量化**：使用 Embedding 模型将文本块转化为高维向量。
- 存储**：将向量及其元数据写入向量数据库，同时保留原始文本以供生成阶段引用。

检索模块

- 查询理解**：对用户输入进行意图识别、查询改写或扩展。
- 向量检索**：在向量空间中进行近似最近邻搜索（ANN），召回 Top-K 个相关片段。
- 重排序**：对初步召回的结果进行精细化打分，剔除噪声，保留最相关内容。

生成模块

- 上下文组装**：将重排序后的文档片段按相关性或逻辑顺序拼接，构建 System Prompt。
- LLM 推理**：大模型根据 Prompt 生成回答，通常需配合引用标注、答案格式化等后处理逻辑。

向量数据库

作为 RAG 的“长期记忆”，向量数据库（如 Milvus、Pinecone、Weaviate、Chroma 等）负责高效存储与检索向量数据，支持过滤、元数据查询及混合搜索功能。

3. RAG 的关键技术

向量嵌入（Embedding）

Embedding 是 RAG 的基石。高质量的 Embedding 模型能将语义相近的文本映射到向量空间中相近的位置。当前主流模型包括 BGE、E5、GTE 及 OpenAI 的 Text-Embedding-3 系列。选择模型时需权衡维度、检索精度及推理速度。

相似度检索

常用度量方式包括余弦相似度（Cosine Similarity）、欧氏距离（L2 Distance）及点积（Dot Product）。在工程实现上，通常采用 HNSW、IVF 等索引算法加速检索，在召回率与延迟之间取得平衡。

上下文窗口管理

LLM 的上下文窗口有限，且存在“迷失中间”现象（即模型更关注开头和结尾的信息）。因此，需对检索内容进行截断、摘要或动态选择，确保关键信息位于 Prompt 的显著位置。

重排序（Rerank）

向量检索属于双塔模型，计算快但精度有限。Rerank 通常采用交叉编码器（Cross-Encoder），对 Query-Document 对进行精细打分。虽然计算成本高，但仅对 Top-K 结果操作，能显著提升最终输入的上下文质量。

4. RAG 的优化策略

查询改写（Query Rewriting）

用户查询往往模糊或口语化。通过 LLM 或规则将原始查询改写为更适合检索的形式，如：

- **HyDE（Hypothetical Document Embeddings）**：先让 LLM 生成一个“假设性回答”，再用该回答去检索真实文档，解决 Query 与 Document 的语义空间不一致问题。
- **多查询扩展**：将单一问题拆解为多个子问题，分别检索后合并结果。

混合检索（Hybrid Search）

结合**关键词检索（BM25）**与**向量检索**。关键词检索对专有名词、精确匹配敏感，向量检索对语义理解强。通过 RRF（Reciprocal Rank Fusion）等算法融合两路结果，可大幅提升召回率。

多路召回与上下文压缩

- **多路召回**：除语义检索外，引入知识图谱检索、SQL 结构化查询等，覆盖不同维度的信息需求。
 - **上下文压缩**：使用 LLM 或提取式摘要算法，对长文档进行压缩，仅保留与问题相关的句子，减少 Token 消耗并降低噪声干扰。
-

5. RAG 的应用场景

企业知识库问答

解决内部文档分散、搜索困难的问题。员工可通过自然语言查询公司制度、技术文档、历史项目资料，系统自动汇总并给出带引用的答案。

智能客服

结合产品手册与 FAQ 库，提供 7x24 小时精准服务。相比传统关键词匹配，RAG 能理解复杂意图，处理多轮对话，并基于最新政策更新答案，无需频繁维护知识库。

法律与金融文档检索

在法律尽职调查或金融研报分析中，RAG 可快速定位合同条款、风险点或财务指标，辅助专业人士进行决策，大幅缩短信息获取时间。

医疗辅助诊断

基于权威医学指南与文献库，为医生提供诊疗建议、药物相互作用查询及最新研究成果，但需严格设置安全护栏，确保输出合规。

6. RAG 面临的挑战与未来趋势

当前挑战

- **幻觉与错误归因**：即使有检索，模型仍可能忽略上下文或错误拼接信息。
- **检索质量瓶颈**：分块粒度不当、Embedding 模型领域适应性差、长尾问题召回不足。
- **长文档处理**：当相关证据分散在多个长文档中时，上下文窗口与推理能力面临考验。
- **评估困难**：缺乏统一的端到端评估指标，检索准确率与生成忠实度往往难以兼得。

未来趋势

- **Agentic RAG**：引入 Agent 架构，使系统具备规划、工具调用与自我反思能力，能主动迭代检索策略。
- **多模态 RAG**：支持图表、音频、视频的统一检索与生成，打破文本模态限制。
- **GraphRAG**：结合知识图谱，增强对实体关系、全局摘要及复杂推理的支持。
- **端侧与轻量化**：随着小参数模型与量化技术进步，RAG 将更多在边缘设备或本地部署，兼顾隐私与响应速度。
- **自适应 RAG**：系统根据问题复杂度动态选择检索策略（如简单问题直接生成，复杂问题多轮检索），实现效率与效果的最优平衡。

RAG 技术正处于从“可用”向“好用”跨越的关键期。随着检索算法、模型能力及系统架构的持续演进，RAG 将成为连接大模型与真实世界知识的标准基础设施，深刻重塑人机交互与信息获取的方式。